From Actions to Algorithms to Policies

A unified exposition of CDT, EDT, FDT/LDT/TDT, UDT 1.0, and UDT 1.1

Draft chapter section

May 2026

Abstract

This chapter section presents a staged development of decision theory as it appears in AI alignment discussions: from causal and evidential action selection, to functional or logical identification with an abstract decision procedure, to updateless reasoning, and finally to explicit policy selection. The central organizing claim is that the historical sequence can be understood as three changes in what the agent takes itself to be choosing. First, FDT/LDT/TDT shift the target of choice from a local physical act to the output of an abstract computation. Second, UDT 1.0 removes ordinary Bayesian updating from the decision rule and evaluates the local output against the prior. Third, UDT 1.1 notices that the local output is still the wrong object in some problems; the correct object is an entire policy, a mapping from observations to actions.

The exposition uses a common formal language inspired by Vanessa Kosoy's shortform formalism: a problem has an event or history space, observations, actions, policies, a utility function, and several possible counterfactual kernels. CDT, EDT, FDT-action, UDT 1.0, and UDT 1.1 then differ by three parameters: the outermost object optimized over, whether the agent conditions on its current observation, and the kind of counterfactual dependence used. The motivating examples are Newcomb's problem, the smoking lesion, counterfactual mugging, and Wei Dai's copied-agent anti-coordination problem.

Contents

1 Orientation: three shifts in the target of choice

Decision theory asks for a rule which, given a description of a decision problem, selects the best available act. In standard philosophical presentations, the central dispute is often framed as a dispute about counterfactuals: should the agent evaluate an action by looking at its causal consequences, as in causal decision theory (CDT), or by conditioning on the hypothesis that it performs that action, as in evidential decision theory (EDT)? In AI alignment, this question becomes sharper. An advanced artificial agent might be copied, simulated, predicted, self-modify, reason about its own source code, bargain with other agents that can inspect it, or act in environments that contain mathematical correlates of its computation. In those settings it is no longer obvious what counts as "the action" whose consequences should be evaluated.

The development from CDT through UDT 1.1 can be organized around three changes.

1. **From physical instantiation to algorithmic identity.** CDT and EDT usually evaluate a local physical act: this hand takes one box, this body smokes, this program returns this output at this time. FDT, LDT, and TDT instead ask what follows from the abstract decision procedure having a certain output. This is the move from identifying with a physical instantiation to identifying with a computation, a policy, or an algorithm.

2. **From updateful to updateless evaluation.** A TDT-style local logical decision rule may still condition on the current observation and ask what is best in this branch. UDT 1.0 instead asks what output would have had the best prior expected consequences, before conditioning on the observation that now seems fixed.

3. **From local output selection to policy selection.** UDT 1.0 still asks for the best output at the current input. Wei Dai's UDT 1.1 fixes a bug in that local formulation by first selecting an entire input-output mapping, and only then applying the selected mapping to the actual input.

This is a stylized history. It is not a clean taxonomy of all uses of the names. In particular, the FDT paper presents the normative slogan "treat one's decision as the output of a fixed mathematical function" and notes that the authors' preferred formulation iterates over policies

rather than actions; later expositions sometimes classify policy-valued FDT as very close to UDT 1.1. For the purposes of exposition, I will use *FDT-action* or *local LDT* for the action-valued logical-counterfactual rule, and *FDT-policy/ UDT 1.1* for the policy-valued rule. This separates the three conceptual moves that are otherwise easy to conflate.

2 A shared language for the paradoxes

Kosoy's shortform formalism starts with common structure shared by several decision theories: an event space, a policy space, a loss or utility function, and events expressing that the agent's behavior is consistent with a policy. Different decision theories then add different extra structure. FDT adds logical counterfactuals over policies; CDT restricts to formally causal counterfactuals; EDT works in extensive form and conditions on following a policy at a decision point. I will use a slightly simplified utility-maximizing version of this idea, designed to make the CDT-to-UDT sequence easy to compare.

Definition 1 (Finite decision frame)**.** *A finite decision frame consists of:*

- *a finite set* Ω *of complete histories* ω*;*
- *a prior distribution* $\mu \in \Delta(\Omega)$*;*
- *an observation map* $X : \Omega \to \mathcal{X}$*;*
- *an action map* $A : \Omega \to \mathcal{A}$*;*
- *a utility function* $U : \Omega \to \mathbb{R}$*;*
- *a policy set* $\Pi \subseteq \mathcal{A}^{\mathcal{X}}$*, where a policy* π *maps each possible observation* $x \in \mathcal{X}$ *to an action* $\pi(x) \in \mathcal{A}$*.*

When mixed policies matter, replace $\mathcal{A}^{\mathcal{X}}$ *by stochastic kernels* $\mathcal{X} \to \Delta(\mathcal{A})$*. For the examples below, deterministic policies suffice.*

A decision theory also needs a way of evaluating counterfactual alternatives. We will use four families of kernels. A kernel is a map from a hypothetical choice to a distribution over histories. These kernels are not all primitive in every theory; the notation is a bookkeeping device that lets us compare them.

Definition 2 (Four counterfactual/evaluative kernels)**.** *Fix an observation* $x \in \mathcal{X}$ *and an action* $a \in \mathcal{A}$*.*

- $K^{\mathsf{Phys}}_{x,a}$ *is the* physical causal intervention *kernel: the distribution over histories obtained by holding the relevant pre-action physical facts fixed and intervening so that the local act at* x *is* a*.*
- $K^{\mathsf{Evid}}_{x,a}$ *is the* evidential *kernel: the factual prior conditional on observing* x *and taking action* a*, i.e.* $\mu(\cdot \mid X = x, A = a)$*, when this conditional is defined.*
- $K^{\mathsf{Log}}_{x,a}$ *is the* local logical *kernel: the distribution over histories under the logical counterfactual that the abstract decision computation, when run on input* x*, outputs* a*.*
- K^{Pol}_{π} *is the* policy logical *kernel: the distribution over histories under the logical counterfactual that the abstract decision computation outputs the entire policy* π*.*

The local logical kernel is the one that creates the most difficulty. In Newcomb-like problems, it is meant to capture dependencies between the agent's present output and a predictor's earlier simulation of that same output. In copied-agent problems, it may also capture dependencies between the current output and another copy's output. In mathematical or proof-based formulations, this is sometimes written using a logical conditional or a proof search over consequences of the proposition "this algorithm outputs a". In graphical FDT-style presentations, it is represented by intervening on a logical node rather than a physical action node. The exact general theory of such logical counterfactuals remains unsettled; the point of this chapter section is to show why one wants such a notion and why the object of the counterfactual changes over time.

Given these kernels, the main decision rules can be written in parallel.

Definition 3 (Decision rules in the shared notation)**.** *Let x be the observation currently received by the agent.*

$$\mathrm{CDT}(x) \in \arg\max_{a\in\mathcal{A}} \mathbb{E}_{K^{\mathsf{Phys}}_{x,a}}[U], \tag{1}$$

$$\mathrm{EDT}(x) \in \arg\max_{a\in\mathcal{A}} \mathbb{E}_{K^{\mathsf{Evid}}_{x,a}}[U], \tag{2}$$

$$\mathrm{LDT}_{\mathrm{local}}(x) \in \arg\max_{a\in\mathcal{A}} \mathbb{E}_{K^{\mathsf{Log}}_{x,a}}[U \mid X = x], \tag{3}$$

$$\mathrm{UDT1}(x) \in \arg\max_{a\in\mathcal{A}} \mathbb{E}_{K^{\mathsf{Log}}_{x,a}}[U], \tag{4}$$

$$\mathrm{UDT1.1}(x) = \pi^*(x), \qquad \pi^* \in \arg\max_{\pi\in\Pi} \mathbb{E}_{K^{\mathsf{Pol}}_{\pi}}[U]. \tag{5}$$

The table below summarizes the three dimensions.

Rule	Outermost object optimized	Update status	Dependence/counterfactual type
CDT	local physical action	updateful	causal intervention on action
EDT	local physical action as evidence	updateful	conditional/evidential dependence
TDT / local LDT / FDT-action	local algorithm output	usually updateful in the historical TDT presentation	logical/subjunctive dependence
UDT 1.0	local algorithm output	updateless	logical/subjunctive dependence
UDT 1.1 / FDT-policy	whole policy	updateless	logical/subjunctive dependence over policies

The word "updateful" means that the rule evaluates options from within the current information state. The word "updateless" means that the rule does not discard branches of the prior merely because the agent now knows it is not in them. UDT is not ignorant of the observation; rather, it uses the observation only after selecting the action or policy according to a prior evaluation. UDT 1.0 still selects a local action at the current input. UDT 1.1 first selects a global input-output mapping.

Remark 1 (What "fair" means here)**.** *A fair decision problem, in this exposition, is one whose dependence on the agent is explicit in the world description. The problem is not allowed to say "CDT agents are punished" or "FDT agents are rewarded". It may contain a predictor, a simulator, a lesion, a copy, or an institution, but these must interact with the agent through its action, source code, policy, or observable behavior. Equivalently, a fair problem should be representable as a world program that takes some representation of the agent or its policy as input and then computes outcomes. This is why Newcomb, smoking lesion, counterfactual mugging, and the copied-agent coordination problem are pedagogically useful: each isolates a specific kind of dependence.*

3 Part 1: CDT and EDT

CDT and EDT are the standard contrast case. Both are action-selection theories. Both ask what to do from the current information state. They disagree about how to evaluate the hypothesis "I take action a". CDT treats a as a causal intervention. EDT treats a as evidence.

In ordinary one-shot decisions where the agent's action is not correlated with hidden facts except through ordinary causal pathways, CDT and EDT often agree. If pressing a button causes a reward, both press. If smoking causes cancer and no other correlations matter, both avoid smoking if the cancer risk is sufficiently high. The conflict appears when action and outcome are correlated in ways that are not downstream of the local act.

3.1 Newcomb's problem: CDT loses predictive dependence

Example 1 (Newcomb's problem)**.** *There are two boxes. The transparent box contains $k = \$1{,}000$. The opaque box contains $M = \$1{,}000{,}000$ iff Omega predicted that the agent would take only the opaque box. The agent may one-box (O) or two-box (T). Omega is highly reliable and has already made the prediction before the agent acts.*

In the policy-response representation, with one observation $\star$, there are two policies:

$$\pi_O(\star) = O, \qquad \pi_T(\star) = T.$$

Assuming perfect prediction for simplicity, the policy-level values are:

Policy π	Omega's prediction	Utility under K_π^{Pol}
π_O	one-box	M
π_T	two-box	k

EDT asks what the act is evidence for. If Omega is accurate, the event "I one-box" is strong evidence that the opaque box is full, and the event "I two-box" is strong evidence that it is empty. So EDT one-boxes. A logical decision theorist also one-boxes, but for a different reason: it evaluates the logical output of the decision procedure, not the evidential news carried by the physical act.

CDT, by contrast, asks what happens if the agent physically takes one box or two while holding fixed the already-filled state of the boxes. In each possible box state, taking both boxes yields exactly k more than taking only the opaque box. So the CDT intervention kernel satisfies

$$\mathbb{E}_{K^{\mathsf{Phys}}_{\star,T}}[U] = \mathbb{E}_{K^{\mathsf{Phys}}_{\star,O}}[U] + k,$$

and CDT two-boxes. CDT is locally dominant with respect to the physical act, but globally poorer with respect to the predictable policy.

This is the first half of the CDT/EDT lesson: some environments depend on what the agent is like, or on what its decision computation will output, without being downstream of the present physical act. CDT deliberately ignores such dependence.

3.2 The smoking lesion: EDT mistakes evidence for control

Example 2 (Smoking lesion)**.** *Smoking gives a benefit $b > 0$. A lesion L causes cancer with cost $C \gg b$ and also makes the agent more likely to smoke. Smoking itself does not cause cancer. The agent may smoke (S) or abstain (N). The decision procedure's chosen policy does not alter whether the lesion is present.*

The policy-level values are simple. Since the lesion distribution is independent of the chosen policy, smoking adds benefit b in both lesion and no-lesion worlds:

$$\mathbb{E}_{K^{\mathsf{Pol}}_{\pi_S}}[U] = \mathbb{E}_{K^{\mathsf{Pol}}_{\pi_N}}[U] + b.$$

Thus CDT smokes, and a logical decision theorist should also smoke, provided the lesion is not itself a logical determinant of the decision algorithm's output.

EDT conditions on the action. If the lesion makes smoking much more likely, then

$$\mathbb{P}(L \mid A = S) \gg \mathbb{P}(L \mid A = N).$$

EDT can therefore evaluate smoking as bad news about cancer:

$$\mathrm{EU}_{\mathrm{EDT}}(S) = b - C\mathbb{P}(L \mid A = S),$$
$$\mathrm{EU}_{\mathrm{EDT}}(N) = -C\mathbb{P}(L \mid A = N).$$

For large enough C and strong enough correlation, EDT abstains even though abstaining does not reduce the probability of cancer. EDT has mistaken a symptom for a lever.

The smoking lesion is often discussed with complications: perhaps the agent observes an urge to smoke; perhaps a refined evidential theory conditions on the urge rather than on the action; perhaps the causal graph should include a deliberation node. Those refinements matter in a full philosophical treatment. For the present historical role, the example isolates the second half of the CDT/EDT lesson: action can be evidence of a hidden variable without being a way to control it.

3.3 The first synthesis pressure

Newcomb and the smoking lesion put opposite pressure on action-selection rules.

Problem	CDT	EDT
Newcomb	two-boxes and predictably gets k	one-boxes and gets M
Smoking lesion	smokes and gets benefit b without changing cancer risk	may abstain because smoking is bad evidence

The natural thought is that CDT is right to ignore merely evidential correlations, while EDT is right to attend to correlations between one's decision and a predictor's earlier behavior. FDT/LDT/TDT attempt to capture exactly this distinction. The relevant dependence is not ordinary evidence, and it is not ordinary physical causation. It is logical or subjunctive dependence: the predictor, the copy, or the institution depends on the same abstract computation that is now being run.

4 Part 2: FDT, LDT, and TDT

FDT, LDT, and TDT are names for a family of logical decision theories. The common slogan is: choose as if you are choosing the logical output of your decision algorithm. This changes the agent's self-location. The agent is not merely this physical hand about to move. It is an implementation of a mathematical procedure, and other parts of the world may be implementations, simulations, predictions, or consequences of that same procedure.

4.1 The local logical rule

The local logical rule can be written as

$$\mathrm{LDT}_{\mathrm{local}}(x) \in \arg\max_{a \in \mathcal{A}} \mathbb{E}_{K^{\mathsf{Log}}_{x,a}}[U \mid X = x].$$

The kernel $K^{\mathsf{Log}}_{x,a}$ says: consider the counterfactual in which the abstract decision computation, on input x, outputs a. If Omega predicted that computation, Omega's prediction changes with it. If another copy runs the same computation on the same input, that copy's output changes with it. If a proof search can show that another program's behavior is linked to this program's output, that behavior changes with it.

This immediately solves the first pair of examples in the intended way. In Newcomb, the logical counterfactual "my decision algorithm outputs one-box" also changes Omega's prediction, so the opaque box is full. In the smoking lesion, the logical counterfactual "my decision algorithm outputs smoke" does not change whether the lesion is present, so the agent smokes.

A useful slogan is:

> CDT evaluates what follows from this act being physically different. EDT evaluates what follows from this act being evidence. FDT/LDT/TDT evaluate what follows from the decision computation being different.

4.2 Parfit's hitchhiker as a clean FDT win

Example 3 (Parfit's hitchhiker)**.** *You are stranded in the desert. A driver will rescue you only if she predicts that, once safely in the city, you will pay her \$100. Rescue is worth \$10,000 to you. After being rescued and arriving in the city, you are asked whether to pay.*

At the city observation $x = \text{city}$, CDT refuses: paying now causally costs \$100 and does not causally affect the already-completed rescue. A local logical decision theorist pays: under the logical counterfactual that the city-output of this decision algorithm is "pay", the driver's earlier prediction is also "pay", so the rescue occurs. The relevant policy values are:

Policy π	Driver prediction	City action	Ex ante utility
π_P	pay	pay	$10{,}000 - 100$
π_R	refuse	refuse	0

This example shows why logical identification is attractive for embedded agents. The agent's current output is not merely a late physical event; it is the thing the predictor was predicting. More precisely, the predictor and the agent are linked by their common relation to the agent's decision computation.

4.3 The next failure: counterfactual mugging

Logical identification by itself is not enough. The remaining question is whether the agent should evaluate the local output from the current observation, or from the prior. Counterfactual mugging isolates this issue.

Example 4 (Counterfactual mugging)**.** *Omega flips a fair coin. If the coin lands tails, Omega does not contact you; instead, if Omega predicted that you would pay in the heads branch, Omega gives you reward* $R = \$10{,}000$*. If the coin lands heads, Omega asks you to pay cost* $c = \$100$*. You observe that you are in the heads branch and must decide whether to pay.*

Let H be the observation "Omega asks for payment" and T be the observation "no request; possible reward". The relevant policies are:

Policy π	Action at H	Utility if H	Utility if T
π_P	pay	$-c$	R
π_N	refuse	0	0

The prior expected utilities are

$$V(\pi_P) = \frac{1}{2}(R - c), \qquad V(\pi_N) = 0.$$

For $R > c$, the ex ante best policy is to pay in the heads branch.

But an updateful local logical rule evaluates from within H:

$$\mathbb{E}_{K^{\mathsf{Log}}_{H,P}}[U \mid X = H] = -c, \qquad \mathbb{E}_{K^{\mathsf{Log}}_{H,N}}[U \mid X = H] = 0.$$

So it refuses. It agrees that its decision algorithm is what Omega predicted, but it has conditioned away the tails branch in which the benefit of being predictable would have appeared. The agent says, in effect, "I now know I am not in the rewarded branch, so only the cost remains."

This is the motivation for updatelessness. The problem is fair because the payment is not rewarded by magic. Rather, Omega's earlier behavior depends on the policy that the agent implements. The policy "pay if asked" has positive prior value; the current branch is just the branch in which that policy pays its cost.

5 Part 3: UDT 1.0 and the updateless move

UDT 1.0 keeps the local logical object but evaluates it against the prior rather than the posterior. In our notation:

$$\mathrm{UDT1}(x) \in \arg\max_{a \in \mathcal{A}} \mathbb{E}_{K^{\mathsf{Log}}_{x,a}}[U].$$

The observation x is still used to determine which local output is under consideration. The difference is that expected utility is computed before discarding branches inconsistent with x.

5.1 Solving counterfactual mugging

In counterfactual mugging, UDT 1.0 at H asks whether the local output "pay at H" or "refuse at H" has better prior consequences. The logical counterfactual that the algorithm outputs pay at H is also the counterfactual under which Omega rewards the tails branch. Thus

$$\mathbb{E}_{K^{\mathsf{Log}}_{H,P}}[U] = \frac{1}{2}(R - c), \qquad \mathbb{E}_{K^{\mathsf{Log}}_{H,N}}[U] = 0,$$

so UDT 1.0 pays. The updateless step can be described in several equivalent ways:

- Do not optimize only for the branch you now know you are in.
- Choose the output that would have been best for the decision computation to have, evaluated before learning which branch you occupy.
- Behave as you would have wanted your earlier self, or your source code, to commit to behaving.

This step also explains why UDT pays in Parfit's hitchhiker and one-boxes in Newcomb. Those are already cases where an earlier predictor responds to the policy or computation the agent implements. Updatelessness says that the agent should not forget the value of being the kind of agent that gets predicted favorably, merely because it has arrived at the branch where the cost is due.

5.2 Updatelessness is not ignorance

A common misunderstanding is that an updateless agent ignores observations. It does not. The observation still matters as an argument to the policy or local decision function. What the agent avoids is using the observation to remove from consideration the branches whose outcomes depended on the agent's policy.

In a normal decision problem with no predictor, no copies, and no branch-crossing dependence, UDT and ordinary expected utility maximization agree. Suppose an agent observes a weather report and chooses whether to carry an umbrella. The policy "carry an umbrella iff rain is likely" is exactly the kind of observation-dependent mapping selected by UDT 1.1, and UDT 1.0's local action evaluation will normally coincide with ordinary conditional expected utility. Updatelessness is not a refusal to learn; it is a refusal to treat the current information state as the only locus of value when the current decision computation has prior consequences across information states.

5.3 The remaining bug: the local output is still too small

UDT 1.0 still optimizes over a local output. This is enough for Newcomb and counterfactual mugging, where the important dependence is between one local output and a predictor's response to that output. It is not enough when the agent must coordinate different outputs of the same policy across different observations.

The key question is: when I counterfactually set $\mathsf{Alg}(x) = a$, what happens to $\mathsf{Alg}(x')$ at another possible observation $x' \neq x$? UDT 1.0 does not explicitly choose the whole function $\mathsf{Alg} : \mathcal{X} \to \mathcal{A}$. It chooses one value of that function and hopes that the logical correlations between different values will fall out correctly. Wei Dai's UDT 1.1 post presents a small copied-agent problem showing that this hope fails.

6 Part 4: UDT 1.1 and policy selection

6.1 The copied-agent anti-coordination problem

Example 5 (Two copied agents must choose differently)**.** *Omega copies you. One copy observes index* 1*, the other observes index* 2*. Each copy must choose* A *or* B*, without communication. Both copies have the same utility function: each receives* 10 *iff the two copies choose different actions, and* 0 *otherwise.*

Here $\mathcal{X} = \{1, 2\}$, $\mathcal{A} = \{A, B\}$, and there are four deterministic policies:

Policy π	$\pi(1)$	$\pi(2)$	Utility
π_{AA}	A	A	0
π_{BB}	B	B	0
π_{AB}	A	B	10
π_{BA}	B	A	10

The correct policy-level answer is obvious: choose either π_{AB} or π_{BA}, with a shared tie-breaking rule, and then each copy applies the same chosen policy to its own observation.

UDT 1.0, however, asks a local question. Suppose the copy observing 1 considers outputting A. What is the logical implication of

$$\mathsf{Alg}(1) = A$$

for

$$\mathsf{Alg}(2)?$$

There are two tempting answers, and both are bad.

- If $\mathsf{Alg}(1) = A$ implies $\mathsf{Alg}(2) = A$, then by symmetry $\mathsf{Alg}(1) = B$ implies $\mathsf{Alg}(2) = B$. Both local choices look like losing choices.
- If $\mathsf{Alg}(1) = A$ implies $\mathsf{Alg}(2) = B$, then by symmetry $\mathsf{Alg}(1) = B$ implies $\mathsf{Alg}(2) = A$. Both local choices look equally good to the copy observing 1. But the copy observing 2 runs analogous reasoning, and with a symmetric tie-breaking rule the copies can still choose the same action.

The problem is not that the agent has the wrong utility function. It is that the counterfactual query has the wrong type. The good object is not the proposition that one entry of the policy table takes one value. The good object is the whole table.

6.2 The UDT 1.1 rule

UDT 1.1 replaces local action selection with policy selection:

$$\pi^* \in \arg\max_{\pi \in \Pi} \mathbb{E}_{K_\pi^{\mathsf{Pol}}}[U], \qquad \text{UDT1.1}(x) = \pi^*(x).$$

In the copied-agent problem, UDT 1.1 compares the four policies directly:

$$V(\pi_{AA}) = 0, \quad V(\pi_{BB}) = 0, \quad V(\pi_{AB}) = 10, \quad V(\pi_{BA}) = 10.$$

If the shared tie-breaker selects π_{AB}, then the copy observing 1 chooses A and the copy observing 2 chooses B. Both copies run the same global policy-selection computation; they merely apply it to different observations.

This is the third shift. FDT/LDT/TDT moved from physical action to algorithm output. UDT 1.0 moved from posterior evaluation to prior evaluation. UDT 1.1 moves from local output to global policy.

6.3 Why policy selection is not just precommitment

UDT 1.1 is often explained as choosing the policy one would have precommitted to before seeing the observation. That is a good intuition, but it can mislead. UDT 1.1 is not adding an extra precommitment action to a standard decision theory. It is changing the object-level decision rule so that the agent's present act is the application of a policy already selected by the same reasoning that evaluates all branches.

This matters for reflective stability. A CDT agent may wish, before entering Newcomb's problem, to self-modify into a one-boxer. A CDT agent in the city in Parfit's hitchhiker may wish it had become the sort of agent who pays. UDT-style agents try to remove this instability by making the later act already answerable to the earlier policy evaluation. In alignment terms, this is one reason decision theory appears in agent foundations: a powerful self-modifying system should not systematically wish that its future decision rule were different in cases it can foresee.

6.4 What UDT 1.1 buys, and what it leaves open

UDT 1.1 handles the examples above by choosing the right level of object. But it is not a finished theory of rational agency. It leaves at least five families of questions.

1. **Logical counterfactuals.** What exactly is K_π^{Pol}? In deterministic settings, the proposition "this algorithm outputs π" may be false for all but one π. How should an agent reason about counterpossible mathematical hypotheses?

2. **Logical uncertainty.** A real agent does not know all consequences of its source code. Running longer changes its mathematical beliefs, but changing beliefs looks like updating, and updatelessness was introduced partly to avoid the wrong kind of updating.

3. **Policy space.** What counts as an available policy? A table from observations to actions? A program? A bounded computation? A distribution over programs? UDT 2 and policy-selection-with-logical-inductors explore variants of this question.

4. **Observation individuation.** What counts as the input x? If the agent's observation includes too much internal state, policy selection can become too late; if it includes too little, the selected policy may be too coarse.

5. **Multi-agent fairness and bargaining.** When agents have different source codes and different utility functions, merely selecting policies with logical correlations does not by itself specify a fair bargaining solution. Program equilibrium, open-source game theory, Nash bargaining variants, and safe Pareto improvements live beyond this first section.

These open questions do not erase the conceptual progress. They explain why the UDT 1.1 endpoint is a useful chapter boundary rather than a final destination.

7 A compact derivation of the staged development

The four main examples can now be arranged as a derivation.

Stage 0: action selection is under-specified

CDT and EDT both optimize over actions. They differ about the evaluative kernel:

$$\text{CDT: } K^{\mathsf{Phys}}_{x,a}, \qquad \text{EDT: } K^{\mathsf{Evid}}_{x,a}.$$

Newcomb shows that K^{Phys} misses predictive dependence. Smoking lesion shows that K^{Evid} overreacts to mere evidence. The missing category is dependence on the decision computation.

Stage 1: identify with the algorithm

FDT/LDT/TDT introduce $K^{\mathsf{Log}}_{x,a}$, the logical counterfactual over the algorithm's output. This handles Newcomb, smoking lesion, and Parfit's hitchhiker in the intended way:

$$\text{local LDT}(x) \in \arg\max_{a\in\mathcal{A}} \mathbb{E}_{K^{\mathsf{Log}}_{x,a}}[U \mid X = x].$$

But the conditional $\mid X = x$ is still updateful.

Stage 2: stop conditioning away the branches whose payoffs depend on the policy

Counterfactual mugging shows that updateful logical action selection can refuse to pay a cost that was worth committing to ex ante. UDT 1.0 removes the conditioning:

$$\text{UDT1}(x) \in \arg\max_{a\in\mathcal{A}} \mathbb{E}_{K^{\mathsf{Log}}_{x,a}}[U].$$

This pays in counterfactual mugging. But it still selects a local output.

Stage 3: select the policy

The copied-agent anti-coordination problem shows that local output selection leaves other inputs of the same algorithm unspecified. UDT 1.1 instead chooses

$$\pi^* \in \arg\max_{\pi\in\Pi} \mathbb{E}_{K^{\mathsf{Pol}}_{\pi}}[U], \qquad \text{then returns } \pi^*(x).$$

This is the clean policy-selection endpoint.

Example	Failure revealed	Shift motivated
Newcomb	physical causation is too local	identify with decision computation
Smoking lesion	evidence is not control	distinguish logical dependence from evidential correlation
Counterfactual mugging	current-branch evaluation is too late	evaluate from the prior
Copied-agent anti-coordination	local output is too small	optimize over policies

8 Further paradoxes in the same notation

This section is not necessary for the CDT-to-UDT 1.1 derivation, but it is useful for a book chapter because many named paradoxes differ mainly by which kernel they stress.

8.1 XOR blackmail

In XOR blackmail, an accurate predictor sends a letter iff exactly one of the following is true: the agent has termites, or the agent would pay upon receiving the letter. Termites cause a large loss L; paying costs c. The letter is correlated with the absence or presence of termites in a way that depends on the agent's policy, but the policy does not cause termites.

Let $T \in \{0, 1\}$ denote termites and $P \in \{0, 1\}$ denote the policy's paying on letter. The letter event is

$$\text{Letter} = T \oplus P.$$

At the observation $x = \text{Letter}$, a naive evidential thought may be "if I pay, that is evidence I do not have termites." But the policy-level values are:

Policy	Worlds where letter arrives	Payment cost
pay-on-letter	$T = 0$	pays in no-termite worlds
refuse-on-letter	$T = 1$	never pays

The agent cannot control T; it can control whether the predictor's letter arrives in termite worlds or no-termite worlds. Refusing makes the letter evidence of termites but avoids being exploitable. This is a clean example of the slogan: decisions are not for changing every correlated fact; they are for choosing which global consistency pattern your algorithm participates in.

8.2 Transparent Newcomb variants

Transparent Newcomb variants add observations about box contents. The agent may see that the opaque box is empty or full before choosing. These cases stress observation individuation and zero-probability conditioning. In the shared language, the policy set includes different actions at observations $x = \text{full}$ and $x = \text{empty}$:

$$\pi : \{\text{full}, \text{empty}\} \to \{O, T\}.$$

An updateful rule may treat the observed box state as settled and choose the locally dominant act. A policy rule asks which mapping from visible box states to acts gives the best prior performance under the predictor's response. Some transparent variants are stable and pseudocausal; others create ill-posed EDT conditionals or unstable policy incentives. They therefore belong naturally after the basic UDT 1.1 exposition, once the reader is comfortable distinguishing local action, local output, and policy.

8.3 Program equilibrium and open-source games

Open-source game theory can be represented by letting a policy be a program and letting K_π^{Pol} include the other players' program analyses of π. In a transparent prisoner's dilemma, for example, the other agent may cooperate iff it proves that your submitted program cooperates with it. The policy object is no longer merely a table from observations to actions; it is source code available for inspection. This is why the program-equilibrium literature is a natural continuation of UDT 1.1 rather than a replacement for it. UDT 1.1 tells us to optimize over policies; open-source game theory studies environments in which policies are themselves strategic artifacts.

9 Conclusion

The path from CDT to UDT 1.1 is a path of increasingly global self-identification.

CDT identifies with a local physical act. That is appropriate when the only relevant dependence is downstream causation, but it fails in predictor problems. EDT identifies the local act as evidence. That captures Newcomb-like correlations, but it also mistakes mere symptoms for levers. FDT/LDT/TDT identify with the output of an abstract decision computation. That captures the dependence between an agent and its predictors, simulations, and copies. UDT 1.0 then evaluates that output from the prior, preserving the value of policies whose benefits occur in branches the agent may not currently occupy. Finally, UDT 1.1 selects an entire policy, solving cases where different possible observations must be coordinated with each other.

The resulting endpoint is not "the final decision theory". It is a conceptual platform. It says that, for embedded agents in Newcomb-like environments, the primary object of rational choice is not a late physical twitch, nor a piece of evidence, nor even a single output of a computation. It is a policy: a structured way the agent's computation responds across possible observations, evaluated by the global consequences of that computation being what it is.

References

[1] Vanessa Kosoy. "Vanessa Kosoy's Shortform," comment on a formalism for decision theory, LessWrong, accessed May 5, 2026. https://www.lesswrong.com/posts/dPmmuaz9szk26BkmD/vanessa-kosoy-s-shortform?commentId=yLwtjqeTCnSNnEXbX

[2] Eliezer Yudkowsky and Nate Soares. "Functional Decision Theory: A New Theory of Instrumental Rationality." arXiv:1710.05060, submitted 2017, revised 2018. https://arxiv.org/abs/1710.05060

[3] Benjamin A. Levinstein and Nate Soares. "Cheating Death in Damascus." *The Journal of Philosophy* 117(5):237–266, 2020. https://doi.org/10.5840/jphil2020117516

[4] Nate Soares and Benja Fallenstein. "Toward Idealized Decision Theory." arXiv:1507.01986, 2015. https://arxiv.org/abs/1507.01986

[5] LessWrong Wiki contributors. "Timeless Decision Theory." LessWrong, last updated February 21, 2025, accessed May 5, 2026. https://www.lesswrong.com/w/timeless-decision-theory

[6] Wei Dai. "Explicit Optimization of Global Strategy (Fixing a Bug in UDT1)." LessWrong, February 19, 2010. https://www.lesswrong.com/posts/g8xh9R7RaNitKtkaa/explicit-optimization-of-global-strategy-fixing-a-bug-in

[7] AlephNeil. "What is Wei Dai's Updateless Decision Theory?" LessWrong, May 19, 2010. https://www.lesswrong.com/posts/5WEoM3RCxN2cQEdzY/what-is-wei-dai-s-updateless-decision-theory

[8] riceissa. "Comparison of decision theories (with a focus on logical-counterfactual decision theories)." AI Alignment Forum, March 16, 2019. https://www.alignmentforum.org/posts/QPhY8Nb7gtT5wvoPH/comparison-of-decision-theories-with-a-focus-on-logical

[9] Abram Demski. "Policy Selection Solves Most Problems." LessWrong, December 1, 2017. https://www.lesswrong.com/posts/5bd75cc58225bf067037550c/policy-selection-solves-most-problems

[10] Abram Demski. "Conceptual Problems with UDT and Policy Selection." LessWrong, June 28, 2019. https://www.lesswrong.com/posts/9sYzoRnmqmxZm4Whf/conceptual-problems-with-udt-and-policy-selection

[11] LessWrong Wiki contributors. "Logical decision theories." LessWrong, accessed May 5, 2026. https://www.lesswrong.com/w/logical-decision-theories

[12] Rob Bensinger. "Decisions are for making bad outcomes inconsistent." Machine Intelligence Research Institute, 2017; LessWrong linkpost excerpt accessed May 5, 2026. https://www.lesswrong.com/posts/9BYo6Q9qBMXWLjqPS/miri-decisions-are-for-making-bad-outcomes-inconsistent

[13] Caspar Oesterheld. "Program equilibrium and open-source game theory – an annotated bibliography." Last updated February 7, 2025, accessed May 5, 2026. https://www.andrew.cmu.edu/user/coesterh/AnnotatedProgEqBibliography.html

[14] Vojtech Kovarík et al. "Game Theory with Simulation of Other Players." arXiv:2305.11261, 2023/2024. https://arxiv.org/html/2305.11261v2